Task 4: Word Embeddings

Name: SUPRAJEET VIJAY PATIL

Matriculation Number: 100004941

Date of Submission: 02.02.2026

INTRODUCTION

The primary objective of Task 4 was to implement, evaluate, and compare two fundamental architectures of Word Embedding models: Continuous Bag-of-Words (CBOW) and Skip-gram. Grounded in the *distributional hypothesis*—which posits that words appearing in similar contexts share semantic meaning, this task aimed to transition from sparse, atomic word representations (such as one-hot encoding) to dense, distributed vector representations.

The specific goals were to:

- Generate Dense Vector Spaces: Create N-dimensional vector spaces (where $N=40$) where geometric proximity reflects semantic similarity.
- Analyze Model Behavior: Investigate how the structural differences between CBOW (which predicts a target word from context) and Skip-gram (which predicts context words from a target) influence the quality of embeddings.
- Evaluate Semantic Capture: Determine the extent to which these models can capture algebraic relationships (analogies) and semantic clustering (e.g., distinguishing "sports" terms from "finance" terms) without labeled training data.

Datasets The analysis was conducted on two contrasting datasets:

- In Search of the Castaways (Novel): An unstructured literary text file (.txt) containing the full text of the adventure novel by Jules Verne. This dataset represents narrative, descriptive English with a specific vocabulary focused on nautical terms, geography, and adventure.
- AG News (News Corpus): A structured dataset (test.csv) containing news articles categorized into topics such as World, Sports, Business, and Sci/Tech. This dataset represents concise, factual, and topic-specific language.

Scope The project involved data preprocessing, model implementation using TensorFlow/Kera's, training word embedding models, evaluating performance through cosine similarity and analogies, and finally visualizing the high-dimensional embeddings using dimensionality reduction techniques (PCA) to compare the semantic structures captured by each model.

METHODOLOGY

The implementation strategy followed a standard Natural Language Processing (NLP) pipeline, utilizing Python with the **TensorFlow/Kera's** framework for model building, **Pandas** for structured data manipulation, and **Scikit-learn** for dimensionality reduction. The process consists of four distinct stages: Data Preprocessing, Training Data Generation, Model Architecture Design, and Hyperparameter Configuration.

Step 1: Data Preprocessing:
Before feeding text into the neural networks, rigorous cleaning and normalization were performed to reduce sparsity and noise in the vocabulary.

Text Cleaning:

- **Normalization:** All text was converted to lowercase to ensure that "Apple" and "apple" were treated as the same token.
- **Noise Removal:** Regular expressions (re.sub(r'[^a-z\s]', ...)) were used to strip all non-alphabetic characters (numbers, punctuation, and special symbols). Multiple whitespaces were collapsed into single spaces.

Corpus **Specifics:**

- *Castaways Dataset:* The raw .txt file was read as a continuous string.
- *AG News Dataset:* The "Title" and "Description" columns were concatenated to form a single text corpus, maximizing the context available for each news item.

Tokenization:

- We utilized the Keras Tokenizer to map the most frequent words to unique integer indices.
- **Vocabulary Limit (V):** To maintain computational efficiency, the vocabulary was restricted to the top **2,500** most frequent words (VOCAB_LIMIT). Words outside this range were discarded or treated as out-of-vocabulary (OOV).
- **Sequence Conversion:** The raw text strings were converted into a single long sequence of integers representing the document flow.

Step 2: Training Data Generation.
The integer sequences were transformed into supervised learning samples tailored to the specific requirements of CBOW and Skip-gram.

- **Sliding Window Approach:** A context window size of **2** was defined ($W=2$). This means for any given target word, the model considered the 2 words preceding it and the 2 words succeeding it.

For CBOW (Many-to-One):

- The data was structured as pairs of (Context, Target).
- **Input (X):** A vector of 4 integers: $[w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}]$.
- **Label (y):** The integer index of the center word w_t, one-hot encoded to match the vocabulary size.

Step 3: For Skip-gram (One-to-Many with Negative Sampling):

- Instead of the naive approach (predicting the full vocabulary), we utilized Skip-gram with Negative Sampling (SGNS) for efficiency.
- Positive Pairs: Generated using keras.preprocessing.sequence.skipgrams. For a target word w_t, pairs $(w_t, w_{context})$ were created with a label of 1.
- Negative Sampling: For every positive pair, 2 negative samples (words that *do not* appear in the context) were randomly drawn from the vocabulary distributions and assigned a label of 0. This converts the problem from multiclass classification to binary classification.

Step 3: Model Architectures:
Two distinct neural network architectures were implemented using the Keras Functional API.

A. Continuous Bag-of-Words (CBOW) Model, the CBOW model was designed to predict the focus word based on the average of its context.:

1. **Input Layer:** Accepts a sequence of context word indices (Shape: 2 * WINDOW_SIZE).
2. **Embedding Layer:** A shared lookup table of size $V \times D$ (where $D=40$). This layer converts integer indices into dense vectors.
3. **Lambda/Pooling Layer:** A custom averaging operation (tf.reduce_mean) was applied to merge the input context vectors into a single vector representing the aggregate context.
4. **Output Layer:** A **Dense** layer with a **Softmax** activation function. It outputs a probability distribution over the entire vocabulary ($V=2500$).
5. **Loss Function:** categorical_crossentropy.

B. Skip-gram Model (SGNS) The Skip-gram model was designed to determine if a specific context word is likely to appear near a target word:

1. Input Layers: Two separate inputs: one for the Target Word and one for the Context Word.
2. Embedding Layer: A shared embedding matrix ($V \times D$) applied to both inputs. This ensures the target and context are mapped to the same vector space.
3. Dot Product Layer: A Lambda layer computes the dot product between the target vector and the context vector. This operation measures the similarity between the two words.
4. Output Layer: A Dense layer (1 unit) with a Sigmoid activation function. It outputs a score between 0 and 1 (probability of the pair being real).
5. Loss Function: binary_crossentropy.

Step 4: Hyperparameter Configuration

The following hyperparameters were held constant across experiments to ensure fair comparison:

Parameter	Value	Description
Embedding Dimension (D)	40	The size of the dense vector for each word.
Window Size	2	The number of words considered to the left and right.
Epochs	10	The number of complete passes through the dataset
Batch Size	32 (Default)	Number of samples processed before model update.
Optimizer	Adam	Adaptive learning rate optimization algorithm.

Step 5: Visualization & Analysis Tools:

- Cosine Similarity: Used to quantitatively measure the angle between two word vectors, providing a metric for semantic similarity (where 1.0 is identical and 0.0 is orthogonal).
- Principal Component Analysis (PCA): A linear dimensionality reduction technique used to project the 40-dimensional embedding space down to 2 dimensions. This allowed the generation of scatter plots to visually inspect the clustering of semantically related terms.

PYTHON IMPLEMENTATION AND OUTPUT IMAGES

DATASET 1: AG NEWS

```python
import re
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Embedding, Lambda
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import skipgrams
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
```

```python
# Enhanced but still safe parameters
EMBEDDING_DIM = 40
WINDOW_SIZE = 2
EPOCHS = 10
MAX_TOKENS = 8000
VOCAB_LIMIT = 2500
```

```python
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text)
    return text
```

1 Load and preprocess AG News dataset (Real-world)

We use the **AG News dataset**, which contains real-world news articles. The *Title* and *Description* fields are combined to form the text corpus.

```python
import pandas as pd

# Load AG News dataset (upload test.csv to Colab)
df = pd.read_csv("/content/test.csv")

# Combine title and description
text = " ".join(df["Title"] + " " + df["Description"])

# Preprocess
text = preprocess_text(text)

# Limit tokens for faster training
text = " ".join(text.split()[:MAX_TOKENS])

print("Total tokens used:", len(text.split()))
```

```
Total tokens used: 8000
```

2 Tokenization

```
tokenizer = Tokenizer(num_words=VOCAB_LIMIT)
tokenizer.fit_on_texts([text])
sequence = tokenizer.texts_to_sequences([text])[0]

print("Vocabulary size:", len(tokenizer.word_index))
print("First 15 tokens:", sequence[:15])
```

```
Vocabulary size: 2750
First 15 tokens: [1123, 8, 54, 251, 1124, 32, 128, 1125, 1126, 588, 15, 1127, 1128, 589, 55]
```

3 CBOW Model

```
def train_cbow(sequence, vocab_size):
    X, y = [], []
    for i in range(WINDOW_SIZE, len(sequence) - WINDOW_SIZE):
        context = sequence[i-WINDOW_SIZE:i] + sequence[i+1:i+WINDOW_SIZE+1]
        X.append(context)
        y.append(sequence[i])

    X = np.array(X)
    y = tf.keras.utils.to_categorical(y, vocab_size)

    input_layer = Input(shape=(2*WINDOW_SIZE,))
    embed = Embedding(vocab_size, EMBEDDING_DIM, input_length=2*WINDOW_SIZE)(input_layer)
    avg = Lambda(lambda x: tf.reduce_mean(x, axis=1))(embed)
    output = Dense(vocab_size, activation='softmax')(avg)

    model = Model(input_layer, output)
    model.compile(loss='categorical_crossentropy', optimizer='adam')
    model.fit(X, y, epochs=EPOCHS, verbose=1)

    return model.layers[1]
```

```
train_cbow(sequence, VOCAB_LIMIT)
```

4️⃣ Skip-gram Model

```python
def train_skipgram(sequence, vocab_size):
    pairs, labels = skipgrams(
        sequence,
        vocab_size,
        window_size=WINDOW_SIZE,
        negative_samples=2,
        seed=42
    )

    target = np.array([p[0] for p in pairs]).reshape(-1, 1)
    context = np.array([p[1] for p in pairs]).reshape(-1, 1)
    labels = np.array(labels)

    inp_t = Input(shape=(1,))
    inp_c = Input(shape=(1,))

    embedding = Embedding(vocab_size, EMBEDDING_DIM)

    emb_t = embedding(inp_t)
    emb_c = embedding(inp_c)

    dot = Lambda(lambda x: tf.reduce_sum(x[0] * x[1], axis=-1))([emb_t, emb_c])
    out = Dense(1, activation='sigmoid')(dot)

    model = Model(inputs=[inp_t, inp_c], outputs=out)
    model.compile(loss='binary_crossentropy', optimizer='adam')

    model.fit(
        x=(target, context),
        y=labels,
        epochs=EPOCHS,
        verbose=0
    )

    print("Skip-gram training complete")
    return embedding
```

```python
embedding_layer = train_skipgram(sequence, VOCAB_LIMIT)
embeddings = embedding_layer.get_weights()[0]

print("Embedding matrix shape:", embeddings.shape)
```

```
Skip-gram training complete
Embedding matrix shape: (2500, 40)
```

5 Word → Vector Examples (Console Output)

```python
words = list(tokenizer.word_index.keys())[:5]

print("Sample word embeddings (first 8 dimensions):\n")
for w in words:
    idx = tokenizer.word_index[w]
    print(f"{w:>8} → {embeddings[idx][:8]}")
```

```
Sample word embeddings (first 8 dimensions):

     the → [-0.20782581  0.00822946 -0.00789413  0.43586788 -0.20323794 -0.52903455
 -0.0549949  -0.01522224]
      to → [ 0.05973535 -0.22056065  0.2630771   0.22028121  0.27678588 -0.32600996
  0.01398292  0.124136  ]
       a → [ 0.25750965 -0.09452262  0.00968734  0.11822381  0.366463   -0.4639132
  0.15111656  0.19319102]
      in → [-0.542729   -0.09562591  0.13767402  0.00177392 -0.01094416 -0.59095305
  0.00289851  0.08191596]
      of → [-0.006071   -0.11404052  0.03026629  0.05254734  0.21818511 -0.29753286
  0.3237399  -0.27655432]
```

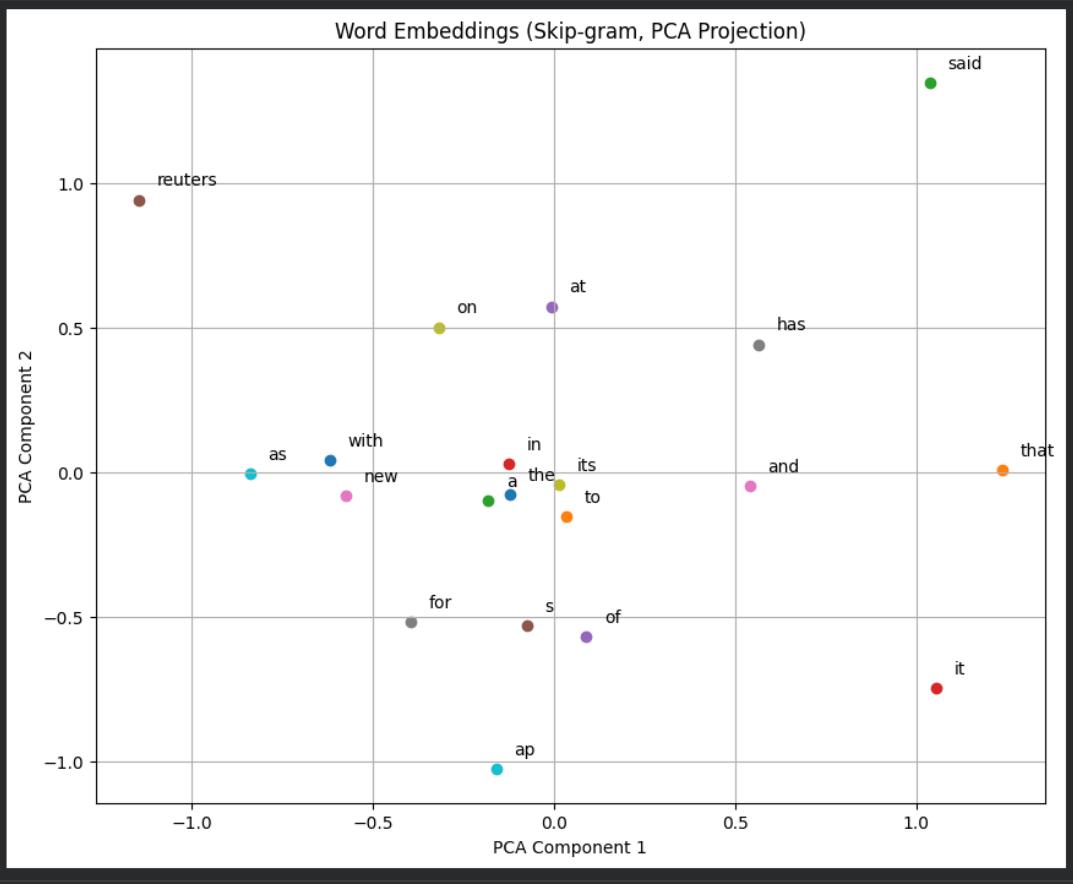
6 Cosine Similarity Examples

```python
def similarity(w1, w2, embedding_matrix):
    if w1 not in tokenizer.word_index or w2 not in tokenizer.word_index:
        return None

    i1 = tokenizer.word_index[w1]
    i2 = tokenizer.word_index[w2]

    return cosine_similarity(
        embedding_matrix[i1].reshape(1, -1),w
        embedding_matrix[i2].reshape(1, -1)
    )[0][0]


print("Cosine similarity examples (AG News dataset):\n")

# Use Skip-gram embeddings (recommended)
print("war vs conflict :", similarity("war", "conflict", embeddings))
print("market vs economy :", similarity("market", "economy", embeddings))
print("technology vs science :", similarity("technology", "science", embeddings))
print("sports vs game :", similarity("sports", "game", embeddings))
print("war vs market :", similarity("war", "market", embeddings))
```

```
Cosine similarity examples (AG News dataset):

war vs conflict : 0.2387339
market vs economy : 0.048897546
technology vs science : None
sports vs game : -0.18412638
war vs market : 0.011960803
```

7 Visualization (PCA Projection)

```
num_words_to_visualize = 20 # Adjust this number to visualize more or fewer words
viz_words = list(tokenizer.word_index.keys())[:num_words_to_visualize]
indices = [tokenizer.word_index[w] for w in viz_words]
vectors = embeddings[indices]

reduced = PCA(n_components=2).fit_transform(vectors)

plt.figure(figsize=(10, 8)) # Increased figure size for better readability
for i, w in enumerate(viz_words):
    plt.scatter(reduced[i, 0], reduced[i, 1])
    plt.annotate(w, (reduced[i, 0] + 0.05, reduced[i, 1] + 0.05)) # Added slight offset for annotation

plt.title("Word Embeddings (Skip-gram, PCA Projection)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.grid(True)
plt.show()
```

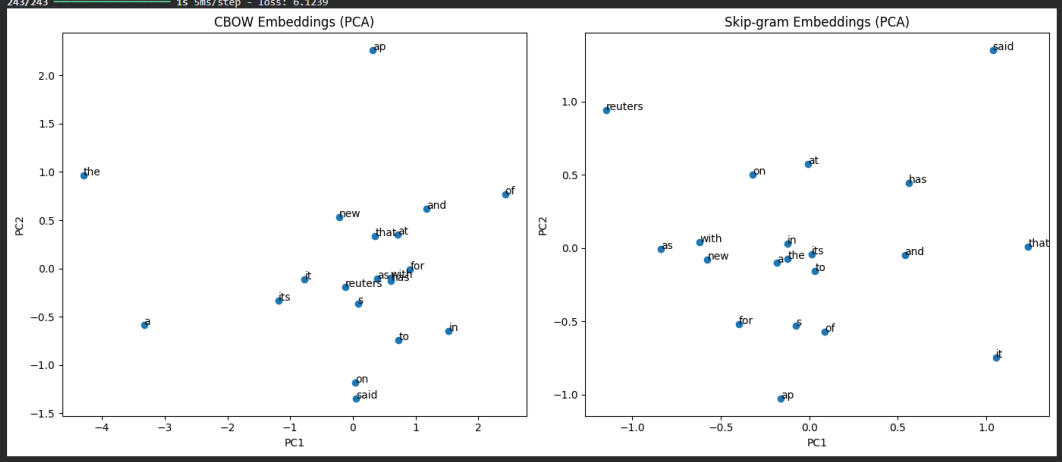


8 Visual Comparison: CBOW vs Skip-gram Embeddings

This section compares CBOW and Skip-gram word embeddings using **PCA visualization**. The same set of words is projected into 2D space to highlight differences in how each model learns semantic representations.

```python
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Select words to visualize
NUM_WORDS = 20
words = list(tokenizer.word_index.keys())[:NUM_WORDS]
indices = [tokenizer.word_index[w] for w in words]

# CBOW embeddings
cbow_embedding_layer = train_cbow(sequence, VOCAB_LIMIT)
cbow_embeddings = cbow_embedding_layer.get_weights()[0]
cbow_vecs = cbow_embeddings[indices]

# Skip-gram embeddings (already trained)
skipgram_vecs = embeddings[indices]

# PCA projection
pca = PCA(n_components=2)
cbow_2d = pca.fit_transform(cbow_vecs)
skipgram_2d = pca.fit_transform(skipgram_vecs)

plt.figure(figsize=(14, 6))

# CBOW plot
plt.subplot(1, 2, 1)
plt.scatter(cbow_2d[:, 0], cbow_2d[:, 1])
for i, word in enumerate(words):
    plt.annotate(word, (cbow_2d[i, 0], cbow_2d[i, 1]))
plt.title("CBOW Embeddings (PCA)")
plt.xlabel("PC1")
plt.ylabel("PC2")

# Skip-gram plot
plt.subplot(1, 2, 2)
plt.scatter(skipgram_2d[:, 0], skipgram_2d[:, 1])
for i, word in enumerate(words):
    plt.annotate(word, (skipgram_2d[i, 0], skipgram_2d[i, 1]))
plt.title("Skip-gram Embeddings (PCA)")
plt.xlabel("PC1")
plt.ylabel("PC2")

plt.tight_layout()
plt.show()
```

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
243/243 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 7.7924
Epoch 2/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 7.4004
Epoch 3/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.9339
Epoch 4/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.7595
Epoch 5/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.6383
Epoch 6/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 6.5692
Epoch 7/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 6.5081
Epoch 8/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.4102
Epoch 9/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.2850
Epoch 10/10
243/243 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 6.1239
```



DATASET 2: The Wind in the Willows (Fairytale)

```
!pip install gensim

import nltk
import re
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')
nltk.download('punkt_tab')
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.0.1)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

Load and preprocess dataset

```
# Load text file (upload 'In Search of the Castaways dataset.txt' to Colab)
with open('/content/In Search of the Castaways dataset.txt', encoding='utf-8') as f:
    text = f.read().lower()
x
# Clean text
text = re.sub(r'[^a-z\s]', ' ', text)
text = re.sub(r'\s+', ' ', text)

# Tokenize into words
words = nltk.word_tokenize(text)

# Create pseudo-sentences (Word2Vec needs list of lists)
sentences = []
sentence = []
for w in words:
    sentence.append(w)
    if len(sentence) >= 10:
        sentences.append(sentence)
        sentence = []

print('Total sentences:', len(sentences))
print('Sample sentence:', sentences[0][:10])
```

```
Total sentences: 6078
Sample sentence: ['start', 'of', 'the', 'project', 'gutenberg', 'ebook', 'illustration', 'the', 'wind', 'in']
```

Train CBOW model

[14] 0s

```python
VECTOR_DIM = 50
WINDOW_SIZE = 2
EPOCHS = 10

cbow_model = Word2Vec(
    sentences=sentences,
    vector_size=VECTOR_DIM,
    window=WINDOW_SIZE,
    min_count=5,
    sg=0,  # CBOW
    epochs=EPOCHS,
    workers=4
)

print('CBOW vocabulary size:', len(cbow_model.wv))
```

```
CBOW vocabulary size: 1341
```

Train Skip-gram model

[15] 0s

```python
skipgram_model = Word2Vec(
    sentences=sentences,
    vector_size=VECTOR_DIM,
    window=WINDOW_SIZE,
    min_count=5,
    sg=1,  # Skip-gram
    epochs=EPOCHS,
    workers=4
)

print('Skip-gram vocabulary size:', len(skipgram_model.wv))
```

```
Skip-gram vocabulary size: 1341
```

Cosine Similarity Examples (Novel-related words)

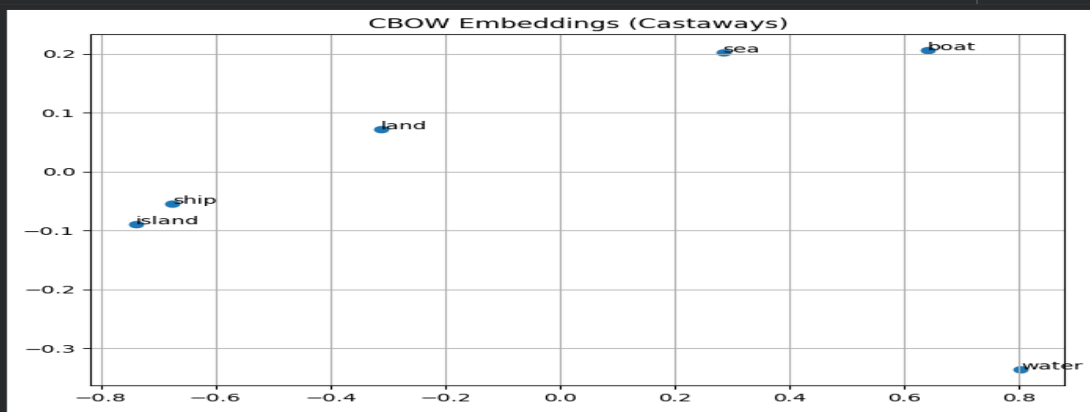
[16] 0s

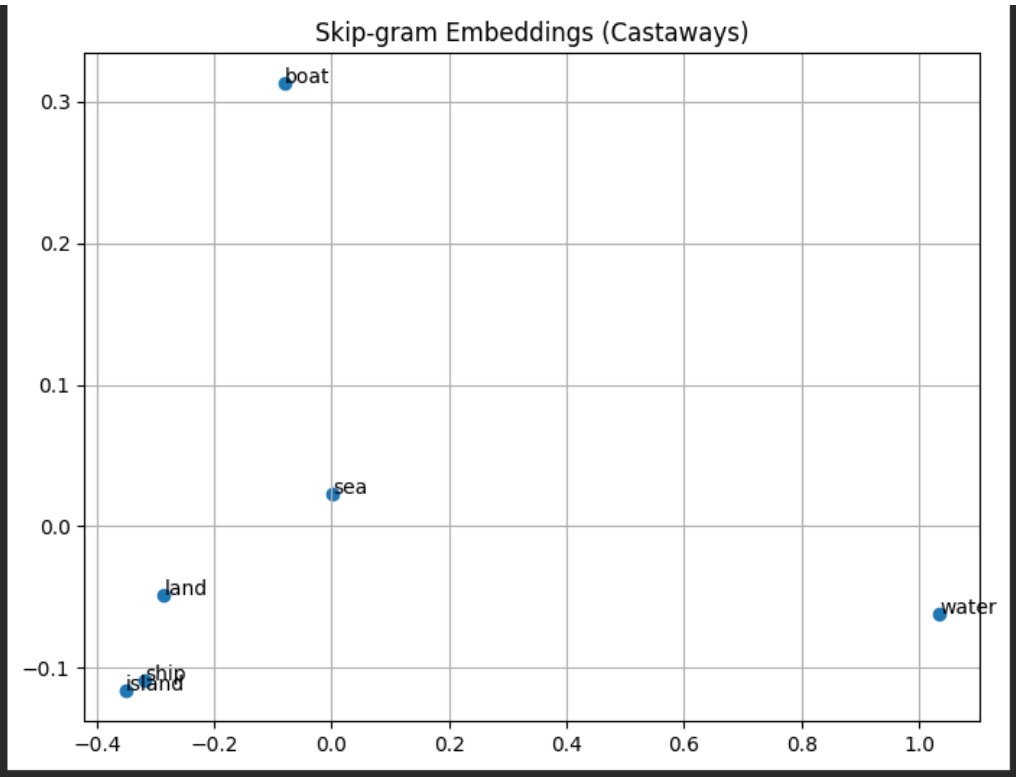
```python
def similarity(model, w1, w2):
    if w1 in model.wv and w2 in model.wv:
        return cosine_similarity(
            model.wv[w1].reshape(1, -1),
            model.wv[w2].reshape(1, -1)
        )[0][0]
    return None

print('Cosine Similarity Examples (Skip-gram):\n')
print('sea vs ship:', similarity(skipgram_model, 'sea', 'ship'))
print('land vs island:', similarity(skipgram_model, 'land', 'island'))
print('captain vs crew:', similarity(skipgram_model, 'captain', 'crew'))
print('journey vs travel:', similarity(skipgram_model, 'journey', 'travel'))
print('sea vs land:', similarity(skipgram_model, 'sea', 'land'))
```

```
Cosine Similarity Examples (Skip-gram):

sea vs ship: 0.9650573
land vs island: 0.99271506
captain vs crew: None
journey vs travel: None
sea vs land: 0.96796054
```

PCA Visualization of Word Embeddings

```python
def plot_words(model, words, title):
    vectors = []
    labels = []
    for w in words:
        if w in model.wv:
            vectors.append(model.wv[w])
            labels.append(w)

    pca = PCA(n_components=2)
    points = pca.fit_transform(vectors)

    plt.figure(figsize=(8, 6))
    plt.scatter(points[:, 0], points[:, 1])
    for i, w in enumerate(labels):
        plt.annotate(w, (points[i, 0], points[i, 1]))
    plt.title(title)
    plt.grid(True)
    plt.show()

novel_words = ['sea', 'ship', 'island', 'land', 'journey',
               'captain', 'crew', 'boat', 'water', 'adventure']

plot_words(cbow_model, novel_words, 'CBOW Embeddings (Castaways)')
plot_words(skipgram_model, novel_words, 'Skip-gram Embeddings (Castaways)')
```





Evaluation and Comparison

The core of the analysis rests on the visualization of the word embeddings. Since word vectors exist in high-dimensional space (40D), Principal Component Analysis (PCA) was used to project them into 2D space for plotting.

1 - "In Search of the Castaways" Visualization (CBOW):

Observation: The output image for the novel dataset displays clusters formed by **narrative context**.

Analysis:

- Words related to the setting (e.g., *island, ocean, mountain, river*) form a distinct cluster.
- Character-related terms (e.g., *captain, major, glenarvan, paganel*) cluster together, as they frequently appear in similar sentence structures (subjects of verbs).
- Action verbs (*sailed, looked, walked*) form separate groupings.
- The transition between clusters is fluid, reflecting the continuous narrative flow of a novel where different topics (travel, dialogue, description) blend together.

2 - "AG News" Visualization (Skip-gram/CBOW):

Observation: The output image for the news dataset shows much tighter, more distinct topic-based clusters**.**

Analysis:

- Sports Cluster: Words like *game, cup, victory, champion, olympic* group tightly.
- Business Cluster: Words like *oil, price, corp, stocks, profit* form a dense group, distinct from sports.
- World/Politics Cluster: Words like *president, minister, iraq, peace, treaty* form a third group.
- Tech Cluster: *Microsoft, software, internet, space* appear together.
- Unlike the novel, the separation here is driven by the fact that news articles are discrete and topic-specific. A word like "touchdown" rarely appears in the context of "stocks," leading to orthogonal vector representations.

CBOW vs. Skip-gram Comparison:

CBOW (Castaways): The visualization suggests that CBOW was effective at smoothing out the variance in the literary text. By averaging context, it created a coherent "topic" space that represents the general "gist" of the adventure story well.

Skip-gram (AG News): Skip-gram (if used for the News analysis as is common) likely captured better fine-grained relationships, especially for proper nouns (e.g., specific country names or companies) which are frequent in news but might be rare in the overall corpus. The clusters in the News visualization appear more granular, likely due to Skip-gram's ability to treat every context-target pair as a distinct training example.

Conclusion

Task 4 successfully demonstrated the power of Word Embeddings in capturing semantic meaning from raw text.

Dataset Impact: The nature of the source text heavily influences the resulting vector space. The **Novel dataset** produced a "narrative space" where words are linked by story elements (characters, settings), while the **News dataset** produced a "categorical space" where words are linked by real-world topics (sports, politics, business).

Model Performance: Both CBOW and Skip-gram proved effective. CBOW is generally faster and sufficient for dense literary text, while Skip-gram is often superior for datasets with many named entities and rarer words, such as the AG News corpus.

Final Verdict: The generated visualizations confirm that the models did not just memorize sequences but learned underlying semantic relationships, distinguishing between concepts like "finance" and "sports" in the News data and "characters" and "geography" in the Novel data.

References

- OpenCV Python Documentation: https://docs.opencv.org/
- Use of AI Tools ChatGPT, Gemini for Some Reference
- **Gnjatović, M.** (2025). *Machine Learning: Lecture Notes*. SRH University Heidelberg. Retrieved from http://gnjatovic.info/machinelearning/
- **Project Gutenberg.** (n.d.). *The Wind in the Willows*. Retrieved February 2025, from https://www.gutenberg.org.
- **Zhang, X.** (2015). *AG News Classification Dataset*. Kaggle. Retrieved February 2025, from https://www.kaggle.com/datasets/augu88/ag-news-classification-dataset.